

Contents

Methodology & Design Decisions	1
Data source decision	1
The melt — two non-obvious choices	1
Train/test split	1
Presence — why a TF-IDF baseline first	2
Presence — why we abandoned retrieval and built MIL	2
The leakage boundary (held everywhere)	2
Significance — testing the small margin instead of hedging about it	2
Controls — testing the two alternative explanations	2
Evaluate the pipeline, not only the parts	3
Risk taxonomy — provenance and validation status	3
Ensemble — parameter-free on purpose	3
Span — focused windows	3
Summarizer — the reference set is the experiment	3
Long-document strategy — why not sparse attention	4
Backbone substitution	4

Methodology & Design Decisions

A short record of *why* the CUAD pipeline was built the way it was — the decisions, the alternatives rejected, and the leakage boundaries held throughout. Companion to `REPORT.md`.

Data source decision

Chose `theatticusproject/cuad` (the raw, multi-file release) over `theatticusproject/cuad-qa` (clean SQuAD) — deliberately, for the preprocessing scope. The raw repo required us to: - discover that `load_dataset()` exposes only PDFs, - pull `master_clauses.csv` and melt it wide→long ourselves, - pull `CUAD_v1.json` separately for character offsets.

This is normal data engineering: different curated files for different needs. The CSV gave us EDA + the summarizer’s clause→summary pairs; the JSON gave full context + span offsets.

The melt — two non-obvious choices

1. **Pair columns by position, not name.** The header `Notice Period To Terminate Renewal-Answer` has a stray space. Position pairing (`text` = odd index, `answer` = even index) is robust to it; string matching is not.
2. **Define “present” from the clause-text list, not the Yes/No answer.** The clause-text cell is a stringified list; non-empty means present. This is uniform across the 8 entity categories (dates, parties) whose `-Answer` holds a value rather than Yes/No, and it yields the clause spans we need anyway.

Train/test split

Contract-level (whole contracts to one side), **408 train / 102 test (80/20)**. Rejected a random row-level split: clauses from the same contract sharing train and test would inflate held-out scores. The split is seeded (`SEED=42`) and reused across all three models so “test” means the same 102 unseen contracts everywhere. Split files exported as `data/train_80.csv` and `data/test_20.csv`.

Presence — why a TF-IDF baseline first

Establishing a strong, cheap, no-length-limit baseline (micro-F1 **0.775**) before any transformer is the single most clarifying decision in the project. It: - reframed every later result as “did the transformer *beat the bar?*” rather than “is 0.7x good?”, - exposed that the head categories are essentially solved by vocabulary alone, - gave us the bar the transformer had to clear — and it did not. The windowed DistilBERT scores 0.694 alone, and the AND-ensemble’s 0.776 only *matches* the baseline rather than beating it (see the significance section below).

Presence — why we abandoned retrieval and built MIL

We **measured the retrieval hit-rate (64%) before training** and saw it capped recall below the baseline. Rather than train a doomed model, we pivoted to sliding-window max-pool (MIL), which removes the ceiling because windows tile the whole document. The discipline — *measure the ceiling, then decide* — saved a 40-minute run.

The leakage boundary (held everywhere)

Allowed (supervision)	Forbidden (leakage)
Use gold offsets to label which training window is positive	Use the answer to <i>build</i> the model’s input
Score retrieval windows by the category/question	Score or pick a window using the gold answer
Fit TF-IDF unsupervised on all windows	Fit any <i>labeled</i> component on test
Max-pool over all windows at inference	Peek at the gold span at inference

The general rule we applied: **labels may shape training targets; they may never shape the input the model sees or the inference-time selection.**

Significance — testing the small margin instead of hedging about it

The AND-ensemble finishing at 0.776 against a 0.775 baseline is a margin of one decision in a thousand on 102 contracts. Rather than caveat it in prose, we measured it with a **cluster bootstrap**: resample the 102 *contracts* with replacement (the 4,182 decisions are not independent — 41 come from each contract and move together), recompute micro-F1 for both models on the same resample, 10,000 times.

The result: the ensemble-vs-baseline difference is **+0.0014, 95% CI [-0.0068, +0.0089]** — the interval contains zero, and the ensemble leads in only 62.6% of resamples. They are statistically indistinguishable. By contrast the baseline-vs-transformer gap is **+0.0805, 95% CI [+0.0626, +0.0992]**, significant in every one of the 10,000 resamples.

The decision this drives: **report the transformer-loses finding as the firm result, and the ensemble as an error-profile improvement rather than a score improvement.** The caution we had already written into the report turned out to point the right way, but it was worth converting into a number.

Controls — testing the two alternative explanations

A bootstrap answers “could this gap be sampling noise?” It cannot answer “was the transformer handicapped?” Two controls were run for that:

Training budget. The baseline is fitted on all 408 contracts; the transformer saw 24,000 of 53,662 windows. Retrained on all 53,662 (140.7 min): micro-F1 moved **-0.0004**, CI [-0.0098, +0.0095]. The gap to TF-IDF held at +0.0809. More data raised recall and lowered precision — the model fires more, not better, which is the max-pooling failure mode. The deficit is architectural.

Random seed. Three runs at the 24,000 budget: transformer SD 0.0085, ensemble SD 0.0022 (range 0.0042). The ensemble’s own seed noise **exceeds its +0.0014 margin** over the baseline — a second, independent reason to call that a tie. The baseline’s lead over the transformer is 9.5 SDs and is seed-robust. Incidentally the ensemble is 4x less seed-variable than the transformer alone, which is a better argument for it than its leaderboard rank.

Design lesson: a margin should be compared against its own noise — from resampling *and* from retraining — before it is reported as a result.

Evaluate the pipeline, not only the parts

Stage-wise scores are measured with the other stages held out of the way, and they do not compose. Run end to end, **21.7%** of gold clauses survive all three stages against the ~64% the component numbers predict (`analysis/e2e.py`). The loss localizes to the max-pooled window being the wrong window 31.7% of the time, and to a train/inference mismatch in the span stage (focus windows re-centred on the answer at training time only). Both are fixable without more parameters. The design lesson: an end-to-end number should be measured before component numbers are quoted as system performance.

Risk taxonomy — provenance and validation status

The 41 assignments were made **by the three of us**, none with legal training, by reasoning about consequences for the weaker party. It is the main contribution over CUAD and it drives what the app tells a user to worry about. **No legal professional has reviewed it as of this writing**, and no validation number appears anywhere in our reporting. The review instrument is ready (`review/risk_taxonomy_review_sheet.csv`). Its virtue is that every assignment is written down with a reason and can be argued with line by line — not that anyone qualified has yet agreed with it.

Ensemble — parameter-free on purpose

We combined TF-IDF and transformer with AND / OR / AVG rules that have **no tunable parameter**, so there is nothing to overfit to the test set. (A weighted or per-category-routed ensemble would likely score higher but would require a validation split to choose weights honestly — listed as future work, not done here.)

Span — focused windows

For training we re-center a 1,200-char window on each gold answer so it fits in 320 tokens and no answer is truncated. At inference the model runs on ordinary windows; the train/inference window distributions differ slightly, an accepted simplification for a laptop build. The standard alternative (doc-stride over the full context, many no-answer features) is the “correct” heavier setup, deferred with the P@80R evaluation.

Summarizer — the reference set is the experiment

We evaluated with **ROUGE-L and a zero-shot-vs-fine-tuned qualitative comparison**, specifically because ROUGE alone is misleading when there are only 41 unique targets — all of which **we wrote ourselves**.

That intuition was later made quantitative by scoring the *same* 150 held-out clauses against a second reference set: clause-specific summaries drafted from the clause text alone, independently of the model (`analysis/summ_pilot.py`).

System	Reference set	ROUGE-L
Fine-tuned	41 category templates	0.775
Fine-tuned	clause-specific	0.116
Zero-shot	clause-specific	0.299

System	Reference set	ROUGE-L
<i>Template alone, no model</i>	clause-specific	0.115

The fine-tuned model emits a template **verbatim on 100% of inputs** (right one 70% of the time), and the bare template with no model matches its score. The design decision this drives: describe the component as **classification-to-template**, never as summarization, in the abstract, results and conclusion. The pilot references are LLM-drafted, not lawyer-authored — independent of the model, but not gold annotation.

Long-document strategy — why not sparse attention

Longformer and friends widen the input window to 4,096 tokens, and the usual one-line dismissal is “too expensive.” That is not the real reason. At ~4 chars/token our median contract is ~8,300 tokens and the longest ~85,000, so a Longformer would still see only **half** a typical contract and a **twentieth** of the largest — we would need windowing and an aggregation rule on top anyway, which is exactly the design we ended up with, only with a far more expensive backbone underneath. Secondary reasons: no distilled Longformer with comparable tooling support, and Clausify needs per-window inference fast enough to feel interactive.

We later measured the cost claim, and it corrected us (analysis/lf_one.py, M4 / 16 GB):

Longformer-base (148.7M), 4,096 tok	Time	Peak memory
forward, batch 1	3.69 s	2.18 GB
training step, batch 1	258 s	14.71 GB
training step, batch 2	2,170 s	20.75 GB

We had written that a 149M-parameter Longformer at 4,096 tokens “exceeds the laptop’s memory budget”. **It does not** — at batch size 1 a full training step fits, in 14.71 GB of 16 GB. The memory claim only holds from batch 2 (past physical memory; 8.4x slower per step from paging). The binding constraint is **time**: 258 s/step makes our two-epoch schedule ~136 days against DistilBERT’s 47 minutes, and a 41-category scan of one median contract takes ~454 s versus ~10.6 s.

The honest status of this section: a design judgement backed by one measurement and one piece of arithmetic, **not** a demonstration that sparse attention would score worse. Nobody trained one. On CUDA hardware that comparison is the fair one to ask for.

Backbone substitution

DistilBERT (not DeBERTa-v3) and FLAN-T5-small (not -base) — forced by Apple MPS instability and laptop wall-clock, **not** a change of method. Every script runs unchanged on CUDA with the larger backbones; the substitution is documented so the numbers are interpreted at the right scale.